

BankOperations

Документация

Изработено от Станил Хаджиев (F113430) и Добри
Германов(F113143).

Съдържание:

Backend.....	1
1. Общ преглед	2
2. Архитектура и Структура	2
3. База данни и Модели (Data & Models).....	2
4. Бизнес логика (Services)	2
5. API Контролери (Controllers)	3
6. Сигурност и Обработка на грешки.....	3
7. Автоматизирано тестване (Unit Testing)	4
Frontend	4
1. Общ преглед.....	4
2. Структура	4
3. Основни компоненти	5
4. Основни файлове.....	5
4.1 index.html (входната точка)	6
4.2 index.js – Стартиране	6
4.3 App.js – Обвивка на приложението.....	6
4.4 api.js – Комуникационен слой	6
4.5 validation.js – Валидация на формуляри	7
4.6 shared.css – Глобални стилове	8
5. Преизползваеми компоненти	8
5.1 Layout.js – Навигационна лента	8
5.2 Alert.js – Временно известие	8
5.3 Pagination.js – Показване на елементи поетапно	8
6. Страници	8
6.1 HomePage.js	8

6.2 ClientsPage.js	8
Всички клиенти	9
Ново физическо лице	9
Ново юридическо лице	9
Детайли за клиент	9
6.3 AccountsPage.js	9
Откриване на сметка	9
Закриване на сметка	10
6.4 CreditsPage.js	10
Отпускане на кредит	10
Детайли за кредит	10
Плащане на вноса	10
7. Интеграционен поток с API	11
8. Изграждане и внедряване	11
Разработка	11
9. Бележки и ограничения	11

Backend

1. Общ преглед

Сървърната част на приложението BankOperations е изградена като RESTful API, използвайки ASP.NET Core (C#). Основната ѝ задача е да обработва HTTP заявките от фронтенда, да прилага бизнес логиката (като изчисляване на кредитни вноски и проверка на банкови баланси) и да съхранява данните сигурно в релационна база данни чрез Entity Framework Core.

2. Архитектура и Структура

Бекендът следва класическата многослойна архитектура, която разделя отговорностите и улеснява поддръжката и тестването:

- **Models (Модели):** Дефинират структурата на данните и връзките между тях.
- **Data (Слой за достъп до данни):** Осъществява връзката с базата данни чрез Entity Framework Core.
- **Services (Бизнес логика):** Съдържа сложните алгоритми и изчисления (напр. генериране на погасителни планове), изолирани от HTTP слоя.
- **Controllers (Контролери):** Обработват входящите HTTP заявки, извикват съответните услуги (services) и връщат HTTP отговори (JSON).

3. База данни и Модели (Data & Models)

Проектът използва Microsoft SQL Server за съхранение на данните. Връзката се управлява от ApplicationDbContext, който картографира (map) C# класовете към SQL таблици.

- **Наследяване (Table-Per-Hierarchy):** Използван е базов клас Client, от който наследяват PhysicalPerson (с допълнителни полета FirstName, LastName, EGN) и LegalEntity (с полета CompanyName, EIK, RepresentativeName).
- **Връзки (Relationships):** Реализирани са връзки тип "Един към много" (One-to-Many). Един Client може да има множество банкови сметки (BankAccount) и кредити (Credit). Всеки Credit притежава списък от вноски (Installment).
- **Прецизност на данните:** Всички финансови полета (баланс, главница, лихва) са изрично дефинирани с тип decimal(18,2), за да се гарантира абсолютна точност до стотинка при банковите изчисления.

4. Бизнес логика (Services)

Сърцето на финансовите изчисления е изолирано в CreditService (имплементиращ ICreditService).

- **Генериране на погасителен план:** Методът GenerateRepaymentPlan изчислява анюитетни (равни месечни) вноски. Той приема сума, продължителност и лихвен

процент (10.5% за потребителски и 4.5% за ипотечен кредит) и използва математическа формула за анюитетен фактор, за да разпредели коректно главницата и лихвата за всеки месец.

- **Плащане на вноски:** При извикване на `PayInstallment`, системата намира съответната вноска по ID и променя нейния статус (`isPaid = true`).

5. API Контролери (Controllers)

Контролерите служат като интерфейс между фронтенда и бекенда. Използват се DTO (Data Transfer Objects) класове (напр. `CreatePhysicalPersonRequest`, `PayInstallmentRequest`) за сигурно приемане на данни.

- **ClientsController:**
 - GET `/api/clients`: Връща списък с всички клиенти.
 - POST `/api/clients/physical`: Създава ново физическо лице.
 - POST `/api/clients/legal`: Създава ново юридическо лице.
 - GET `/api/clients/{id}`: Връща пълни детайли за клиент (Eager Loading на сметки, кредити и вноски).
- **BankAccountsController:**
 - POST `/api/bankaccounts`: Открива нова банкова сметка, като автоматично генерира валиден български IBAN номер.
 - POST `/api/bankaccounts/{id}/close`: Закрива сметка. Бизнес правило: Сметката не може да бъде закрыта, ако по нея има положителен баланс.
- **CreditsController:**
 - POST `/api/credits`: Отпуска нов кредит чрез извикване на `CreditService` и записва генерирания погасителен план.
 - GET `/api/credits/{id}`: Връща детайли за конкретен кредит и неговите вноски, подредени хронологично.
 - POST `/api/credits/pay`: Приема заявка за плащане на конкретна вноска.

6. Сигурност и Обработка на грешки

- **CORS (Cross-Origin Resource Sharing):** В `Program.cs` е конфигурирана политика `AllowFrontend`, която позволява на React приложението (стартирано на различен порт) да комуникира безпроблемно с API-то.
- **GlobalExceptionHandler:** Внедрен е глобален middleware за прихващане на грешки (`Infrastructure.GlobalExceptionHandler`). Ако възникне непредвидено изключение, то

се логва на сървъра, а към клиента се връща стандартизиран отговор (ProblemDetails) с HTTP статус 500, предпазвайки системата от изтичане на чувствителна сървърна информация.

- **Предотвратяване на безкрайни цикли:** Заради двупосочните връзки в базата данни (напр. Клиент -> Кредит -> Клиент), JSON сериализаторът е конфигуриран с `ReferenceHandler.IgnoreCycles`.

7. Автоматизирано тестване (Unit Testing)

Бекенд логиката е покрита с автоматизирани Unit тестове, написани с рамката xUnit.

- За тестването се използва In-Memory Database (база данни в RAM паметта), предоставена от Entity Framework Core. Това позволява тестовете (напр. `ClientsControllerTests`) да се изпълняват изключително бързо и напълно изолирано един от друг, без да модифицират реалната SQL база данни.

Frontend

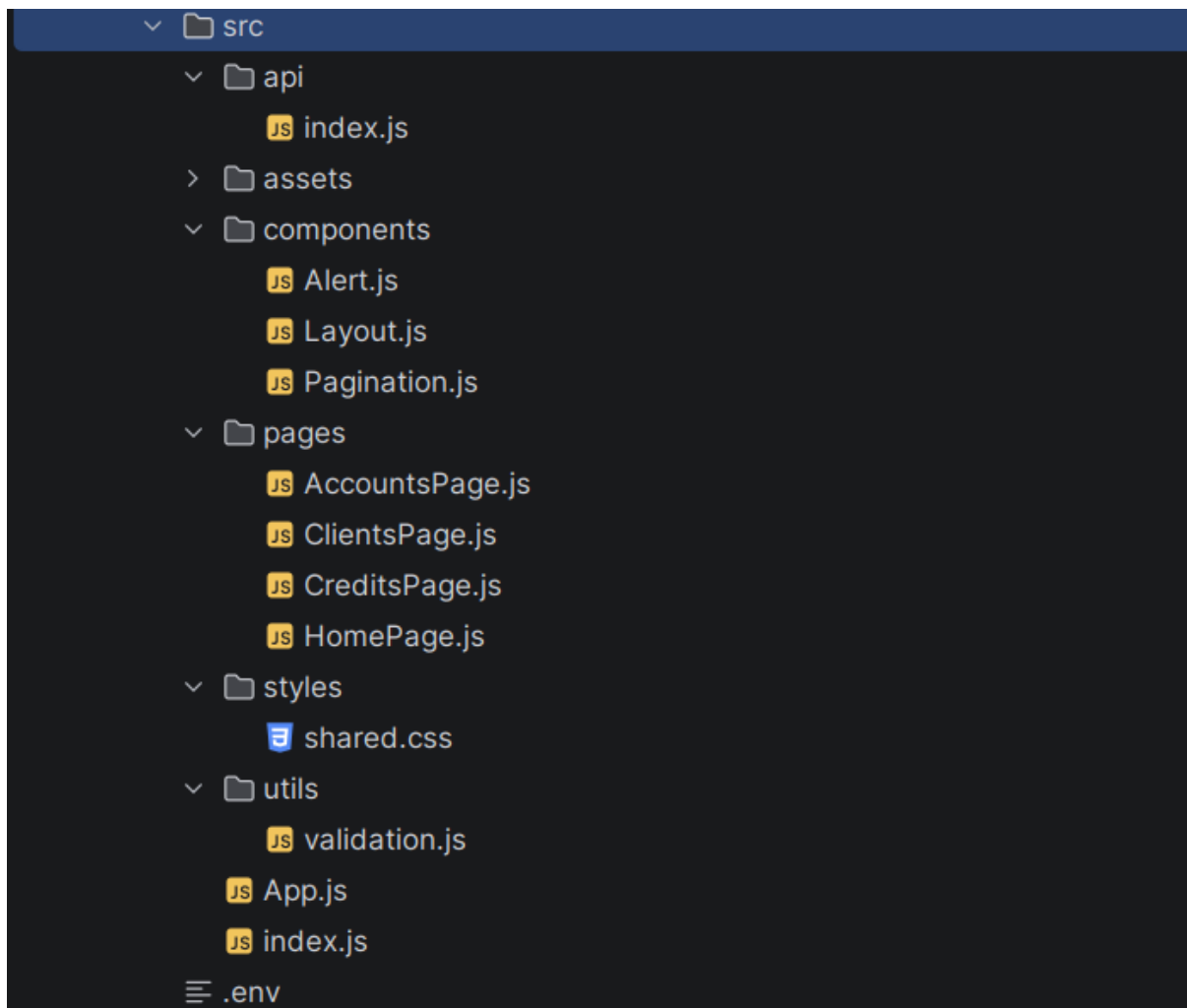
1. Общ преглед

Фронтендът на BankOperations е **React Single-Page Application (SPA)**, която позволява на банкови служители да:

- Управляват клиенти (физически лица и юридически лица)
- Откриват и закриват банкови сметки
- Отпускат кредити, преглеждат погасителни планове и плащат вноски

Фронтендът комуникира с Asp.Net Core бекенд чрез REST API заявки.

2. Структура



3. Основни компоненти

- **Компонентно-базиран** – Всяка UI част е преизползваема функция, която връща HTML-подобен код (JSX).
- **Маршрутизация от страна на клиента** – React Router променя видимата страница без пълно презареждане на брауъра.
- **Състояние** – Всяка страница пази собствени данни (напр. списък с клиенти, стойности на формуляри) и обновява UI при промяна на данните.
- **Без глобално състояние** – Няма Redux или Context; всяка страница извиква API независимо.

4. Основни файлове

4.1 index.html (входната точка)

Цялото React приложение се инжектира един `<div>`, когато JavaScript пакетът се зареди.

4.2 index.js – Стартиране

- Намира елемента `#root`.
- Монтира компонента `<App />`, обвит с `<BrowserRouter>`, за да се позволи навигация.

4.3 App.js – Обвивка на приложението

- Нарисува `<Layout>` (който съдържа `navbar-a`).
- Вътре в `<Layout>`, `<Routes>` определя коя страница да се покаже според URL-а.

```
<Layout>
<Routes>
  <Route path="/" element={<HomePage />} />
  <Route path="/clients" element={<ClientsPage />} />
  <Route path="/accounts" element={<AccountsPage />} />
  <Route path="/credits" element={<CreditsPage />} />
</Routes>
</Layout>
```

4.4 api.js – Комуникационен слой

Този модул е **единственото място**, което комуникира с бекенда. Той експортира девет функции, всяка от които отговаря на бекенд endpoint.

Базов URL - задава се чрез променлива на средата или по подразбиране <http://localhost:5089>.

```
const API = process.env.REACT_APP_API_URL || 'http://localhost:5089';
```

Основна функция `apiFetch`

Приема път, HTTP метод и незадължително тяло.

- Добавя `Content-Type: application/json`.

- Превръща тялото в JSON string.
- Извиква `fetch(API + path)`.
- Обработва JSON отговора.
- При HTTP грешки хвърля `Error` със съобщението от бекенда (напр. "Client not found.").

Експортирани функции (всички използват `apiFetch`):

Функция	Метод	Крайна точка	Действие в бекенда
<code>getClients()</code>	GET	<code>/api/clients</code>	Връща всички клиенти
<code>getClientDetails(id)</code>	GET	<code>/api/clients/{id}</code>	Връща клиент със сметки и кредити
<code>createPhysicalPerson()</code>	POST	<code>/api/clients/physical</code>	Създава физическо лице
<code>createLegalEntity()</code>	POST	<code>/api/clients/legal</code>	Създава юридическо лице
<code>createBankAccount()</code>	POST	<code>/api/bankaccounts</code>	Открива нова сметка
<code>closeBankAccount(id)</code>	POST	<code>/api/bankaccounts/{id}/close</code>	Закрива сметка (ако баланс = 0)
<code>createCredit()</code>	POST	<code>/api/credits</code>	Отпуска кредит + погасителен план
<code>getCreditDetails(id)</code>	GET	<code>/api/credits/{id}</code>	Връща кредит с вноски
<code>payInstallment(id)</code>	POST	<code>/api/credits/pay</code>	Маркира вноска като платена

4.5 validation.js – Валидация на формуляри

Чисти помощни функции, които проверяват въведените от потребителя данни преди извикване на API:

- `isEmpty(val)` – проверява за празен string или само интервали.
- `isValidEmail(val)` – прост regex за email.
- `isDigits(val, len)` – точна дължина цифри.
- `isPositiveInt(val)` – цяло число > 0 .
- `isPositiveNumber(val)` – число > 0 .
- `isNonNegativeNumber(val)` – число ≥ 0 .

Използват се на всички формуляри, за да се избегне изпращане на невалидни данни към сървъра.

4.6 shared.css – Глобални стилове

Тъмна, елегантна тема, използваща CSS custom properties (--bg, --accent и т.н.). Стилизира навигацията, картите, формулярите, табове, таблиците, известията и бутоните. Зарежда се в App.js.

5. Преизползваеми компоненти

5.1 Layout.js – Навигационна лента

- Винаги видима отгоре.
- Съдържа линкове към Начало (/), Клиенти (/clients), Сметки (/accounts), Кредити (/credits).
- Използва NavLink от React Router за подчертаване на активната страница.
- Мобилно хамбургер меню превключва CSS клас.

5.2 Alert.js – Временно известие

- Елементи: message (текст), type (success / danger / warning).
- Автоматично се скрива след 4 секунди.
- Използва се на всяка страница за показване на отговори от API или валидационни предупреждения.

5.3 Pagination.js – Показване на елементи поетапно

- Елементи: page, totalPages, onPageChange.
- Показва бутони „Предишна“ и „Следваща“ с деактивирани състояния.
- Няма връзка с бекенда – работи само върху масиви, вече заредени в паметта.

6. Страници

6.1 HomePage.js

Статични карти, водещи към трите основни секции. Без API извиквания.

6.2 ClientsPage.js

Табовете: Всички клиенти | Ново физическо лице | Ново юридическо лице | Детайли за клиент

Всички клиенти

- Извлича всички клиенти еднократно чрез `getClients()`.
- Показва страницирана таблица (от страна на клиента, по 8 на страница).
- Колони: ID, Email, Телефон.

Ново физическо лице

- Формуляр с валидация (собствено име, фамилия, ЕГН точно 10 цифри, email, телефон).
- Извиква `createPhysicalPerson()`.
- При успех обновява списъка с клиенти и изчиства формуляра.

Ново юридическо лице

- Формуляр (име на фирма, ЕИК точно 9 цифри, представител, email, телефон).
- Извиква `createLegalEntity()`.
- Същият поток като при физическото лице.

Детайли за клиент

- Падащо меню с всички клиенти (изградено от същия `allClients` списък).
- При избор извиква `getClientDetails(id)`.
- Показва:
 - **Информация за клиента** – тип (физически/юридически), име/фирма, идентификатор, контакти.
 - **Таблица с банкови сметки** – ID, IBAN, баланс, статус (Активна/Закрита).
 - **Таблица с кредити** – ID, тип, сума, срок, платени/общо вноски.

6.3 AccountsPage.js

Табовете: Откриване на сметка | Закриване на сметка

Откриване на сметка

- Падащо меню за избор на клиент (заредено от `getClients()`).

- Поле за начален баланс (≥ 0).
- Валидира, след което извиква `createBankAccount()`.
- Показва генерирания IBAN в специално поле.

Закриване на сметка

- Двустъпково каскадно меню:
 - Избери клиент → извиква `getClientDetails(clientId)` за показване само на активни сметки (статус = 0).
 - Избери активна сметка.
- Бутон „Закриване на сметка“ извиква `closeBankAccount(accountId)`.
- Бекендът проверява баланс = 0, задава статус „Закрита“ или връща грешка.
- След затваряне обновява списъка със сметки.

6.4 CreditsPage.js

Табове: Отпускане на кредит | Детайли за кредит | Плащане на вноска

Отпускане на кредит

- Формуляр: клиент, тип кредит (Потребителски = 0, Ипотечен = 1), сума (> 0), срок (1–360 месеца).
- Извиква `createCredit()`.
- Бекендът автоматично генерира всички вноски (анюитетна формула) и връща кредита.
- Показва съобщение за успех с ID на новия кредит.

Детайли за кредит

- Стъпка 1: Избор на клиент → `getClientDetails(clientId)` за списък с кредитите на клиента.
- Стъпка 2: Избор на кредит → `getCreditDetails(creditId)` връща пълния погасителен план.
- Показва обобщение на кредита (тип, сума, лихва, срок, брой платени вноски).
- Показва страницирана таблица с вноски (месец №, общо, главница, лихва, остатък, статус: Платена/Чакаща).

Плащане на вноска

- Четири каскадни падащи менюта:
 - Клиент → зарежда кредитите и активните сметки на клиента.
 - Кредит → зарежда неплатените вноски.
 - Вноска – избери една неплатена.

- Сметка (от която се плаща) – в момента **не се използва** в логиката на бекенда (само UI placeholder).
- При „Плати“: извиква `payInstallment(installmentId)`.
- Бекендът маркира вноската като платена (`isPaid = true`). Не се извършва превод на средства.

7. Интеграционен поток с API

1. Потребителят работи с формуляр.
2. Фронтенд валидация (`validation.js`) проверява въведените данни.
3. Ако са валидни, `api.js` изпраща HTTP заявка към бекенда.
4. Бекенд контролерът обработва заявката, работи с базата данни и връща JSON.
5. `apiFetch` обработва отговора; ако HTTP статусът не е ОК, хвърля грешка със съобщението от бекенда.
6. Страницата улавя грешката и показва известие, или използва успешните данни за обновяване на UI.

8. Изграждане и внедряване

Стартирайте и двете:

- Бекенд: `dotnet run` → стартира на <http://localhost:5089>
- Фронтенд: `npm start` → стартира на <http://localhost:3000>

Фронтендът използва `REACT_APP_API_URL`, за да сочи към бекенда. CORS е разрешен на бекенда за всички origins.

9. Бележки и ограничения

- **Няма автентикация** – Приложението е напълно отворено.
- **Странициране от страна на клиента** – Списъците се страницират само в брауъра, не на сървъра. За големи обеми от данни трябва да се добави сървърно странициране.
- **Плащане на вноски** – Падащото меню „Плати от сметка“ съществува само във фронтенда.
- **Обработка на грешки** – Всички API грешки се улавят и показват, но мрежовите прекъсвания не се обработват специално.